

UNIT 3

1. Conditional Statements

Conditional statements in Python are used to execute certain blocks of code based on specific conditions. These statements help control the flow of a program, making it behave differently in different situations.

Control Flow Statements in Python are fundamental building blocks that dictate the execution order of a program. They enable developers to create logical pathways and make decisions in their code, using structures like if, for, and while.

These statements are essential for adding complexity and functionality to Python scripts, allowing for conditional execution and repetitive tasks. This introduction will briefly explore how these control flow mechanisms enhance the versatility and efficiency of Python programming.

Python has three types of control structures:

- **Sequential**
- **Selection**
- **Repetition**

Importance of Control Statements in Python

Control flow statements are crucial in Python programming, as they dictate the order in which the code is executed, enabling decision-making, looping, and branching in a program. This makes them indispensable for creating dynamic and efficient software. Here's a brief overview of their importance:

1. Control statements like if, elif, and else allow programs to execute different code blocks based on certain conditions.
2. Loops, including for and while, enables the execution of a code block multiple times.
3. By using control flow statements, programmers can write cleaner, more organized code.
4. Control flow in Python allows for creating efficient algorithms that can make decisions and repeat tasks without human intervention.
5. Overall, control flow statements are foundational to programming in Python, enabling the creation of versatile, efficient, and intelligent applications. Their proper use is a cornerstone of good programming practices and is essential for leveraging the full potential of Python.

Types of Control Flow in Python

The control flow statements in Python include:

- The if statement

- The if-else statement
- The nested-if statement
- The if-elif-else chain

The if statement in Python

The if statement is a fundamental control flow in Python used to execute a block of code based on a condition. It allows for conditional execution, where the specified code block runs only if the condition evaluates to True. If the condition is False, the code block is skipped, and the program executes the next line of code outside the if statement.

Syntax of If Statement:

```
if condition:  
# Statements to execute if  
# condition is true
```

Example:

```
x = 10  
if x > 5:  
    print("x is greater than 5")
```

Output:

```
x is greater than 5
```

The if-else statement in Python

The if-else statement in Python is a fundamental control flow mechanism that allows for the conditional execution of code blocks. It enables a program to execute specific actions when a specified condition is true and different actions when the condition is false. This bifurcation based on conditions makes the if-else statement a critical tool for decision-making in programming.

Syntax of Python If-Else:

```
if condition:  
    # code to execute if condition is true  
else:  
    # code to execute if condition is false
```

Example:

```
x = 3
if x > 5:
    print("x is greater than 5")
else:
    print("x is not greater than 5")
```

Output:

```
x is not greater than 5
```

The nested-if statement in Python

The nested-if statement in Python is a control flow structure that allows you to check multiple conditions sequentially, within other if statements. This nesting capability is handy for handling more complex decision-making scenarios where multiple layers of conditions need to be evaluated.

Example:

```
x = 12
if x > 10:
    if x % 2 == 0:
        print("x is greater than 10 and even")
```

Output:

```
x is greater than 10 and even
```

The if-elif-else ladder in Python

The if-elif-else ladder in Python is a series of conditional statements that provide multiple possible execution paths for a program. It starts with an if condition, followed by one or more elif (short for "else if") blocks to check various conditions in sequence, and ends with an optional else block as a catch-all for any conditions not met by the preceding if or elif blocks.

Syntax:

```
if condition1:
    # code to execute if condition1 is true
```

elif condition2:

code to execute if condition2 is true

else:

code to execute if none of the above conditions are true

Example:

```
x = 7
if x > 10:
    print("Greater than 10")
elif x == 7:
    print("x is 7")
else:
    print("Less than 10 and not 7")
```

Output:

```
x is 7
```

Short Hand if statement in Python

The shorthand if statement in Python allows for a condensed form of writing conditional expressions, primarily used for executing a single action or operation when a condition is true, without providing an alternative for the false case. This form is handy for making code more concise when only a single condition needs to be checked to operate.

Syntax:

if condition: statement

Example Consider a scenario where we want to print a message only if a specific condition is met, such as a variable x being greater than 10.

```
x = 15
if x > 10: print("x is greater than 10")
```

Output:

```
x is greater than 10
```

In this example, since x is 15, which satisfies the condition $x > 10$, the print statement is executed, displaying "x is greater than 10". This illustrates the use of a shorthand if statement to perform a quick, conditional action in a single line of code.

Short Hand if-else statement in Python

The shorthand if-else statement in Python, also known as the ternary operator, offers a more concise way to execute simple conditional assignments. This operator allows you to evaluate a condition in a single line, returning one value if the condition is true and another if it's false. It's beneficial for straightforward, inline operations that don't require the full syntax of traditional if-else statements.

Syntax:

statement_if_true if condition else statement_if_false

Example Let's consider a simple example where we want to assign the string "Adult" to a variable if a person's age is 18 or more and "Minor" if it's less than 18.

```
age = 20
status = "Adult"
if age >= 18 else "Minor"
print(status)
```

Output:

Adult

In this example, since age is 20, which satisfies the condition $age \geq 18$, the expression evaluates to "Adult", and thus, the status variable is assigned the value "Adult".

Iteration in Python

In Python, iteration is the process of repeating a block of code a specified number of times. It can be achieved using loops such as the for loop and while loop.

Repeating identical or similar tasks without making errors is something that computers do well and people do poorly. Repeated execution of a set of statements is called iteration. Because iteration is so common, Python provides several language features to make it easier.

Python For Loops

Iterating over a sequence using a for loop in Python (that is, a **list**, a **tuple**, a **dictionary**, a **set**, or a **string**). This functions more like

an **iterator** method found in object-oriented programming languages than the for loop in other programming languages.

Example 1:

```
# Python program to count the number of even integers in the list.  
# List of integers  
integers = [8, 3, 7, 4, 6, 21, 9]  
# variable to store the count  
count = 0  
# iterate over the list  
for val in integers:  
    if val % 2 == 0:  
        count += 1  
print("The count of even numbers is", count)
```

Output:

```
The count of even numbers is 3
```

Example 2:

```
# Python program  
country = ['India', 'UAE', 'United Kingdom']  
for x in country:  
    print(x)
```

Output:

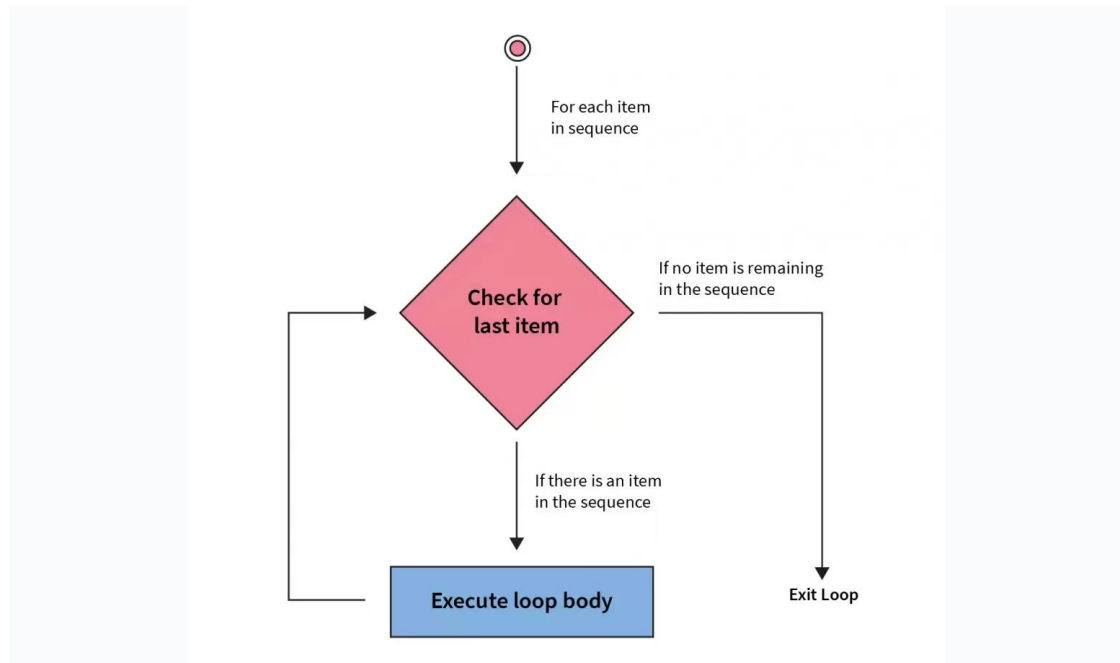
```
India  
UAE  
United Kingdom
```

For Loop Syntax

```
for val in sequence:  
    loop body
```

Above **val** is an iterator variable that takes the value of each item from the sequence on each iteration. After taking the value in **val**, the loop body executes. for loop continues until the last item of the sequence is reached.

Flowchart of for Loop in Python



Looping through a String

In Python, we can loop through a string. The string is an iterable object containing a sequence of characters. Although Strings are immutable, we can perform operations using the characters to get the desired result.

Let's see the above methods in the below code snippet. There are many ways of iterating through the string in Python using a for loop.

- **Method 1:** We can iterate over the string character by character until the string gets exhausted. The for loop in each iteration takes one letter from the string, stores it in the variable character, and then prints it.
- **Method 2:** Another way to iterate over the string is using indexing. Since indexing starts at 0, we can use the range() function, which we have seen above. The range() function will take the stop value as an argument, which serves as the end value for the loop.
- **Method 3:** One more way to iterate over the string is [the enumerate\(\) method](#). The enumerate() takes the string as an argument and returns a key-value pair. The key is the index number by default and the value of the corresponding letter in the string. The key can also be other numbers as a counter, the starting of which can be passed as an argument.

```
# Python program to iterate over a string using a for loop.  
string_var = 'Interviewbit'
```

```

# Method 1: Iterating over a string
for character in string_var:
    print(character, end=' ')
print('\n')

# Method 2: Iterating using indexing
for i in range(len(string_var)):
    print(string_var[i], end=' ')
print('\n')

# Method 3: Iterating using enumerate()
for k, v in enumerate(string_var):
    print(v, end=' ')

```

Output:

```

Interviewbit
Interviewbit

```

The break Statement

The break is a loop control statement. Using the break statement, we can stop the loop or take out the execution control from the loop before its completion. We stop the further execution of the for loop usually based on some condition(s). Whenever the specified condition is encountered, we use a break statement that takes us out of the Python for a loop. However, the break statement can be used without the condition. But with the for loop, it is usually accompanied by condition(s) based on the logic of our program when we want to come out from the loop.

Below is the code snippet that shows how it works. The Python for loop iterates through a list of integers. In each iteration, an integer is taken into the variable 'i' and checked if it is even or not. If the number stored in i is even, then the break statement is executed, and the control comes out of the loop else the odd numbers are printed.

```

# Python program using the break statement
numbers = [5, 9, 7, 13, 8, 1, 11]
for i in numbers:
    # Come out of the loop if an even number is encountered.
    if i%2 == 0:
        print("\nBreaking the Loop. Even Value
Encountered.")

```



```
        break
    print(i, end=' ')
```

Output:

```
5 9 7 13
Breaking the Loop. Even Value Encountered.
```

The continue Statement

The continue is a loop control statement. Using the continue statement, we can stop the current iteration of the loop based on some condition(s), and control again comes to the beginning of the loop with the next iteration. Similar to the break statement, a continue statement does not necessarily have to be accompanied by a condition. The continue statement can be used without condition. But generally, within the for loop in Python, we find its condition application to skip some statements in certain iterations.

In this, we iterate over a list of different country names and print them. But we want that if the name 'China' is encountered, then the name is not printed. For this purpose, we have a condition accompanied by a continue statement that skips the print command and initiates the subsequent iteration of the loop.

```
# Python program with the use of continue Statement
country = ['India', 'China', 'UAE', 'United Kingdom']
for x in country:
    if x == 'China':
        continue
    print(x)
```

Output:

```
India
UAE
United Kingdom
```

In the above code, you can see that when the value of x is equal to China, the loop doesn't execute fully, and the control goes back to the next iteration. So, this was just a simple case of using a continue statement in Python for loops. We can have many other applications of it depending on the program we are making.

The range() Function

range() is a built-in function provided by Python. This function is commonly used with a for loop for looping over a range of numbers. This function returns a sequence of numbers that, by default, starts with zero, increments by 1, and ends at a specified number passed as an argument. However, it is possible to provide this function as an argument for the starting number, increment step size, and the number at which it needs to stop. There are three ways in which a range() function can be used:

- **range(stop):** It takes one argument and is the default version of the range() function. It will produce a sequence of numbers starting from 0, incrementing by 1, and ending at the specified number count. For example, range(15) will generate the numbers starting at 0 and ending at 14.
- **range(start, stop):** This will take as an argument the starting and the ending numbers and will generate the sequence incrementing by 1. For example, range(5, 10) will generate a sequence of numbers starting at 5 and ending at 9, by default incremented by 1.
- **range(start, stop, step/Increment):** This version takes the start value, step value by which the series will increment(positive integer) or decrement(negative integer), and ending values parameters and generates the sequence. For example, range(6, 21, 2) will generate all the even numbers between 6 and 21. There will be an increment of 2 in successive values as the step size is 2.

To iterate through a list, range() is generally used in combination with the **len()** function. Also, while using the range(), we will be accessing the items stored in the sequence by indexing.

Now, let's understand with an example how we can use range() with the for loop in python. In the below code, we are using the for loop to iterate over a list containing the cities using the range() function. Since range() needs to be given at least one argument, i.e., the stop value, the stop value will be the size of the list as we want to print all the cities concatenating with the "I Love" string. So the loop will go through each integer starting from 0 to end (which is the list size) and using indexing, the city name from the list will be printed.

```
#Program to iterate through a list using range()  
city = ['London', 'Paris', 'Mumbai', 'Sydney', 'California']  
size = len(city)  
#iterating over the list using the range() function  
for x in range(size):  
    print("I Love", city[x])
```

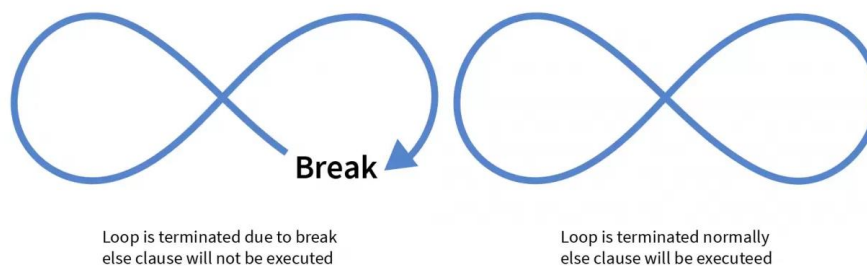
Output:

```
I Love London
```

```
I Love Paris  
I Love Mumbai  
I Love Sydney  
I Love California
```

Else in Python For Loop

Python allows for an optional **else** block in conjunction with a for loop. The code within this else block runs when the for loop completes its iterations. However, this else block is bypassed if the for loop ends prematurely due to a break statement. As a result, the else section is only executed when the loop concludes without encountering a break.



A question might come to your mind instead of using else with if, what is the need for using the else clause with the python for a loop? Imagine you are writing a program to find whether a value is present in a sequence or not. So when the loop exhausts, if the value is not found, the else part will get executed; else, we can break the loop if the value is present in the sequence.

Using the Python for loop, we will be iterating over the list of integers and checking if the item exists or not.

If the item exists, then we print "Item Found", and it comes out of the loop, and the else part doesn't get executed.

If the item is not present in the list, then the loop completes all the iterations and executes the else block. "Item Not Found." will be printed in this case.

```
# Program to find whether a value is present or not in a sequence  
numbers = [5, 9, 10, 7, 13, 3, 20, 11]  
# Search Term  
item = 15
```

```

for val in numbers:
    if val == item:
        print("Item Found.")
        Break
    else:
        print("Item Not Found!")

```

Output:

```
Item Not Found!
```

The application of using else with the for loop in Python is not limited to this, but there may be many other applications like managing the [nested loops in python](#), checking limits or boundaries, etc.

Nested Loops in Python

Python allows the use of a loop within the loop. This is known as **nesting**. For every single iteration of the outer loop, the inner loop executes all its iterations. For instance, if the outer loop has n iterations and the inner loop has m iterations, then the total number of iterations will be n x m. That is for each iteration of the outer loop. The inner loop will execute m times.

Let's see an example below that makes this more clear. In this, we have two lists named names and cities. We want to print each element in the list 'names' with all the items in the list 'city' concatenated in between with the string 'loves' (for example, x loves y). So first, we will be iterating over the list 'names' using the for loop. This will be the outer loop. Now for every iteration of the outer loop, the inner loop will be iterating over the list 'city'. The total number of iterations, in this case, will be the multiplication of the number of elements in both lists.

```

# Python program having nested for loops
names = ['Zeeshan', 'Shubham', 'Vikram']
city = ['Bangalore', 'London', 'Paris']
for i in names:
    for j in city:
        print(f'{i} loves {j}.')

```

Output:

```

Zeeshan loves Bangalore.
Zeeshan loves London.

```

```
Zeeshan loves Paris.  
Shubham loves Bangalore.  
Shubham loves London.  
Shubham loves Paris.  
Vikram loves Bangalore.  
Vikram loves London.  
Vikram loves Paris.
```

So, above, we can see that the total number of times the nested loops are executed is 9. Since $n = 3$ for the outer loop and $m = 3$ for the inner loop. So $n \times m$ is $3 \times 3 = 9$.

The pass Statement

The indentation block of the for loop in Python cannot be left blank. Leaving the body of the for loop blank would result in an error. If for some reason, we are required to leave the for loop body blank, then we use a pass statement. By putting the [pass statement](#), we are just telling the loop to execute normally, and controls come out of the loop after all iterations are completed.

Below is the code snippet that can help us understand this better. In the below code, we want to run a for loop in Python on the string object 'name'. But let's suppose the loop body requires certain functions to be called that we haven't declared and defined yet. And there must be some order and names defined to call those functions. So, while we will be working on the logic of those functions, we can put a pass statement. This will help us keep track of what we need to add something there as well, and if we run our code, then the error will also be avoided.

```
# Python program having an empty loop  
name = 'Interviewbit'  
for alphabet in name:  
    Pass  
print(f'The last letter in the name is {alphabet}')
```

Output:

```
The last letter in the name is t
```

While Loop in Python

Python While Loop is a programming construct used to repeatedly execute a set of **statements** until a **condition** is met. When the condition in the program becomes false, the line immediately after the loop is performed. While loop is classified as an indefinite iteration.

What is While Loop in Python?

While loop statement in **Python** is used to execute statement(s) repeatedly. The number of times the while loop is executed is not known in advance, so the while loop is called an Indefinite Iteration statement. It will keep on repeatedly executing as long as a while condition is true, and it will stop repeating only if the condition becomes false.

you are asked to print your name in Python 100 times. The most naive approach would be to type/copy-paste the statement `print("Name")` 100 times and execute. But that is such a redundant task with 100 lines of Python code.

The best possible approach will be using the while loop statement in Python. And here's how it works: Write a while loop. Place the `print("Name")` inside the while loop. Write a condition so that it fails after executing 100 times, and voila.

This code is at most 4-5 statements, unlike the naive approach, which took 100 statements. So, in other words, a while loop makes our life easy.

Syntax of While Loop in Python

```
while expression:  
    statement(s)
```

Below is an explanation of the components in syntax:

- `statement(s)` can be a single statement or a block of uniformly indented statements. Python treats uniformly indented statements as a block.
- `expression` is a statement in the condition statement in Python, it is evaluated to be True or False. Python interprets all non-zero values as true and 0 or None as false. While loop keeps on repeating as long as the expression returns True. It will stop when the expression returns False
- `while` is the keyword used to write a while loop statement.
- the while expression is the head/starting of the while loop statement, while the `statement(s)` that are uniformly indented make up the body(inside) of the while loop.

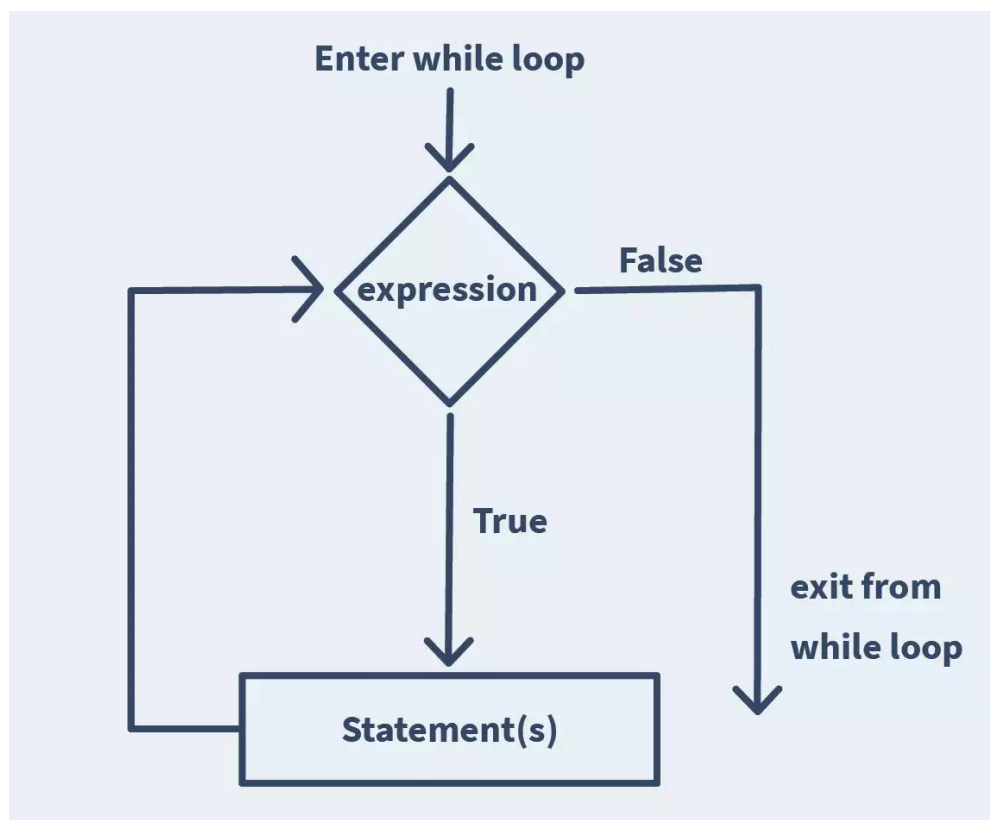
Working of while:

- First, the expression is evaluated. If it returns true, control is passed to the indented `statement(s)` inside the while loop.

- All the statements that are indented to the right below the while loop for the body of the while loop and are executed.
- Once the statements are executed, control is again passed to the expression, and the expression is re-evaluated. If it returns true, the body is executed again. This repeats until the expression returns false.
- Once the while loop break, control passed to the next unindented line of the while loop

We shall explore the working of while loop more clearly with an example in the next sections of the article.

FlowChart of While Loop in Python



Example of Python While Loop

#1 Write a program to print a name ten times in Python using the while loop

Program:

```
count = 0;
while count < 10:
    print("My name is Vidyut")
    count += 1
```

Output:

```
My name is Vidyut
My name is Vidyut
My name is Vidyut
My name is Vidyut
My name is Vidyut
My name is Vidyut
My name is Vidyut
My name is Vidyut
My name is Vidyut
My name is Vidyut
```

Explanation:

- The objective is to print the “My name is Vidyut” string ten times.
- We took a variable count and initialized it to 0
- The expression ‘count < 10’ is evaluated, and since $0 < 10$, control goes inside the while loop
- The ‘print’ statement and count += 1 statement is uniformly indented from the body of the while loop. They are both executed. The string is printed to output once, and the count value increments by 1.
- Control goes back to the expression ‘count < 10’. And since $1 < 10$, control again goes inside the while loop.
- This process is repeated till the count value is incremented to 10, and the print statement has been executed a total of 10 times.
- Now, since the condition $10 < 10$ returns false, the loop breaks, the body of the while loop is not executed, and the control jumps to the next statement after the while statement.

While Loop with else

While statement in python also supports else statement in association with it. The else statement is optional. Below points illustrate the working of while else:

- The else statement written after the while statement will be executed when the expression of while returns False.
- So, the else statement will be executed once the execution of the while is completed normally(By expression returning False).
- If a break statement is executed inside the while loop and the execution is stopped, then else block is skipped, and control jumps directly to the statement below the while..else.
- Notice the difference between the statements inside the else block and the statements written below the while..else.

- Syntax:**

The body of else can be a single statement or a block of uniformly indented statements. The else is executed only once after the while loop condition returns false, and the loop breaks.

```
count = 0;
while count < 10:
    print("My name is Vidyut")
    count += 1
else:
    print("String is printed ten times!")
```

[illegible]

String is printed 10 times!

Explanation:

- The objective is to print the “My name is Vidyut” string ten times.
- We took a variable count and initialized it to 0
- The expression ‘count < 10’ is evaluated, and since $0 < 10$, control goes inside the while loop
- The ‘print’ statement and count += 1 statement, which is uniformly indented from the body of the while loop. They are both executed. The string is printed to output once, and the count value increments by 1.
- Now, control goes back to the expression ‘count < 10’. And since $1 < 10$, control again goes inside the while loop.
- This process is repeated till the count value is incremented to 10, and the print statement has been executed a total of 10 times.
- Now, since the condition $10 < 10$ returns false, the loop breaks, and the body of the while loop is not executed, and control jumps to the next statement after the while statement, which is the else statement.
- Now, the body of else statements gets executed, and the print statement is executed.
- Then, the flow of control goes to the next statement after the while-else statement.

Single Statement While Loop in Python

If there is only one statement in the body of the while loop, the statement can be placed in the same line as the while header statement. This variant is supported in python. The below syntax makes this clear.

```
while expression: statement
```

This is only allowed for a single statement if there are multiple statements, all of them should be written inside the body of the while loop, and that too uniformly indented.

Example:

```
count = 0
while count < 10: print("I run Infinitely")
```

Output:

The String “I run Infinitely” getting printed on the console in infinity

Explanation:

- The count is initialized to 0

- The expression `count < 10` is evaluated, and the single statement inside the while loop body is executed.
- The control then jumps back to the expression `count < 10`, and it again returns true.
- This process keeps on running infinitely, as the expression `count < 10` will never return false since we are not modifying the count value anywhere.
- So this loop can also be called an Infinite while loop. Let us clearly define and understand what an infinite while loop is in the next section.

The Infinite While Loop in Python

The Infinite while loop is a loop that never stops its repeated execution because the expression (condition) defined in the while header will never return False. The example we saw in the above section is one such infinite while loop.

There can be many use cases of Infinite while loop in programming. A simple instance is a system/program that keeps on running without stopping, like a server program. So that the client program can connect to the server program anytime as long as the server program keeps running.

Let us look at an example Infinite while loop program

```
count = 0
while count != 1:
    name = str(input("What is your name? "))
    print("Hi " + name)
```

Output:

```
What is your name? Rahil
Hi Rahil
What is your name? Raavi
Hi Raavi
What is your name? Rejesh
Hi Rejesh
What is your name? Traceback (most recent call last):
  File "main.py", line 3, in <module>
    name = str(input("What is your name? "))
KeyboardInterrupt
```

Explanation:

- Here, the expression `count != 1` is always True. So the body of the while loop is executed on infinite.
- This loop keeps asking for input and outputting the value until we hit CTRL + C, which generates a keyboard interrupt to break the loop.

While Loop Control Statements in Python

Loop control statements/ keywords in python written inside a while loop affect the flow of execution of the while loop. They alter the normal execution of a loop. Below are the loop control statements that are supported in python:

Continue Statement:

Continue statement written inside the body of while loop returns the flow of control back to the expression in the header, skipping the execution of the rest of the statements inside the body.

Example:

```
count = 0
while count < 10:
    count += 1;
    if(count == 5): continue
    print("The value of the count is " + str(count))
```

Output:

```
The value of the count is 1
The value of the count is 2
The value of the count is 3
The value of the count is 4
The value of the count is 6
The value of the count is 7
The value of the count is 8
The value of the count is 9
The value of the count is 10
```

Explanation:

- Count is initialized to 0
- `count < 10` is evaluated, and the body of the while is executed.
- When the value of count is equal to 5. The if statement condition `count == 5` will be true. And the statement inside if which is `continue` is executed.
- Once the `continue` statement gets executed, control jumps directly to the while header again, skipping all the statements below it hence the print statement "The value of count..." is not executed in case of `count == 5`
- After the count becomes six again, the flow of control is similar to the normal while loop.

Break Statement:

Break statement inside the body of a while loop will stop the repeated execution and bring the control out of the while loop. It is like an abrupt force stop of a while loop. If it is a `while..else` statement, then the `else` part is also skipped when a `break` statement is executed. Control jumps directly to the statement below the `while..else` statements.

Example:

```
count = 0
while count < 10:
    count += 1;
    if(count == 5): break
    print("The value of the count is " + str(count))
```

Output:

```
The value of the count is 1
The value of the count is 2
The value of the count is 3
The value of the count is 4
```

Explanation:

- Count is initialized to 0
- `count < 10` is evaluated, and the body of the while is executed.
- When the value of count is equal to 5. The if statement condition `count == 5` will be true. And the statement inside if the `break` is executed.
- Once the `break` statement gets executed, control jumps directly out of the while loop, skipping all the statements below it, and the repeated execution of the loop stops. Hence no values equal to and below five got printed as output.

Pass Statement:

A pass statement can be seen as a dummy/empty statement. It is used to write empty loops, empty blocks, empty functions, classes, etc. It doesn't really affect the flow of control. It is used to keep the flow seamless.

Example:

```
count = 0
while count < 10:
    count += 1;
    if(count == 5): pass
    print("The value of the count is " + str(count))
```

Output:

```
The value of the count is 1
The value of the count is 2
The value of the count is 3
The value of the count is 4
The value of the count is 5
The value of the count is 6
The value of the count is 7
The value of the count is 8
The value of the count is 9
The value of the count is 10
```

Explanation:

- Count is initialized to 0
- count < 10 is evaluated, and the body of the while is executed.
- When the value of count is equal to 5. The if statement condition count == 5 will be true. And the statement inside if which is pass is executed.
- In this example, the flow is not disturbed when a pass gets executed since it is a dummy statement. The if condition gets evaluated without any errors and control and goes to the next statement after the if, which is the print statement.

Fruitful() Functions in Python

Fruitful Python functions are those that yield a result after computation. Unlike void functions, which perform a job without returning a result, productive functions add to a program's overall logic by generating an output. These routines include reusable pieces of code, which improves code modularity and maintainability.

What is a Fruitful Function in Python?

In Python, a fruitful function is like a powerhouse that not only performs a task but also hands back the required piece of information when it's done – like a loop that keeps on giving. Picture it as a function that not only does its job but also brings back a result that you can use elsewhere in your code.

Let's break it down with an example. Consider a function called `multiply_numbers` that takes two parameters, multiplies them together, and returns the result. Here's how it might look in Python:

```
def multiply_numbers(x, y):  
    result = x * y  
    return result
```

Now, you can use this function, passing it two integers, and it will not only execute the multiplication but also return the result:

```
product = multiply_numbers(5, 7)  
print(product)
```

In this scenario, running `multiply_numbers(5, 7)` returns 35, which is kept in the `product` variable. So, a productive function does more than just complete a job; it also returns a result that may be captured and used elsewhere in your code. It's like receiving a two-for-one deal: the function accomplishes its job, and you get a good return.

Dead Code

Dead code in Python refers to lines or blocks of code that exist in a program but are never executed. It's like having a dusty old book on your shelf that you've never opened. On the other hand, a fruitful function is one that not only serves a purpose but also adds value to your code.

Consider this Python example:

```
def calculate_square(number):  
    result = number * number  
    return result
```

```
def calculate_cube(number):
    result = number * number * number
    return result

def main():
    value = 5
    square_result = calculate_square(value)
    print(f"The square of {value} is: {square_result}")
    # The following line is dead code because it's never executed
    cube_result = calculate_cube(value)
    print(f"The cube of {value} is: {cube_result}")

if __name__ == "__main__":
    main()
```

In this example, the `calculate_cube` function is dead code because it is never called by the `main` function. It's like having a recipe for a meal you've never made. Removing dead code is critical for keeping programs clean and efficient.

Example of Fruitful Function in Python

In Python programming, productive functions stand out as captivating magicians that weave magic into our code. Imagine a function that not only performs duties but also bestows wonderful gifts on us; this is the core of a productive function.

Consider this: you construct a function, and instead of simply doing a collection of activities, it gracefully returns a result. That outcome, my buddy, is the product of the function's efforts, and it might range from a simple integer to a complicated data structure.

```
def calculate_square(number):
    square = number ** 2
    return square
```

Here, the function `calculate_square` accepts an integer as input, conducts the magical squaring operation, and kindly returns the squared result. Voila! You've just watched a productive function in action.

Fruitful functions improve code clarity while simultaneously encouraging reusability, making them a useful tool in Python development. So, embrace the magic, unleash the power of useful functions, and watch your code spring to life with elegance and efficiency.

Void Functions

In Python, void functions and fruitful functions perform various roles in programming, enhancing code efficiency and clarity. Let's look at each.

Void functions, as the name implies, do not return anything. They do a series of activities and run commands without creating any results. These functions are critical for code modularization, reusability, and improved code organization.

```
def greet_user(name):  
    print(f'Hello, {name}!')  
# Call the void function  
greet_user("Alice")
```

Fruitful functions, on the other hand, are intended to return a value and provide the caller with information. These routines are very handy when you want a result for further processing or presentation. Here's a straightforward illustration of a useful function that adds two numbers:

```
def add_numbers(a, b):  
    result = a + b  
    return result  
# Call the fruitful function  
sum_result = add_numbers(3, 5)  
print(f'The sum is: {sum_result}')
```

Understanding when to use void or fruitful functions allows programmers to design more modular and efficient code, which improves the clarity and maintainability of their Python programs.

Parameters in Fruitful Functions in Python

In Python, understanding parameters in fruitful functions is key to crafting efficient and dynamic code. Parameters serve as variables that get values when the function is invoked, enabling us to tailor the function's behaviour. Let us go into this essential topic.

In a vast array of fruitful functions, parameters act as input channels, allowing your functions to process a wide range of inputs. They behave as placeholders, anxiously awaiting values when the function is called. Consider a function for calculating the area of a rectangle. Here, length and width are parameters that require particular values to get a valid outcome.

Parameters are defined by declaring them within the function's parentheses and providing their names and types. These inputs convert simple programs into diverse tools capable of handling a variety of datasets. Python's versatility shows through because it enables default parameter values, which adds an added layer of adaptability.

When you call the function, you pass the actual values (arguments) for these parameters. This dynamic interaction of parameters and arguments enables your functions to respond to individual demands, resulting in code that is both functional and highly reusable.

So, understanding arguments in useful functions allows you to write Python code that is both powerful and customizable, making your programming experience more straightforward and efficient.

Function Composition

Function composition in productive functions is a powerful Python programming technique that enables you to combine many functions to build more complicated and reusable code. It's like building blocks for your code, with each function performing a specific task that can be effortlessly integrated to get the required result.

Assume you have two functions, say `multiply_by_two` and `add_five`, each meant to do a certain mathematical operation. Function composition allows you to connect these two functions to form a new function, `combined_function`. This produces a single function that multiplies an input by two and then adds five to the output.

```
def multiply_by_two(x):  
    return x * 2  
def add_five(x):  
    return x + 5  
def combined_function(x):  
    return add_five(multiply_by_two(x))
```

Using function composition makes your code more modular and easier to maintain. You can reuse existing functions to increase code reusability and efficiency in your programs. This method improves code readability and promotes a clean, organized coding style.

So we can say that, function composition simplifies large processes by dividing them into smaller, manageable functions, resulting in a more structured and maintained codebase in Python programming.

Local and Global Scope

Local scope refers to variables defined within a given function. These variables are specific to that function, providing encapsulation and limiting unintentional interaction with other portions of your program. Global scope, on the other hand, refers to variables that are declared outside of any function and hence available across the codebase.

In the context of fruitful functions, local scope becomes pivotal. Variables specified within the function are only valid during its execution, ensuring a controlled

computing environment. When a variable is required outside of the bounds of a given function, global scope is invoked, allowing it to be used across the program.

In simple terms, mastering local and global scope in fruitful functions empowers Python developers to write efficient and organized code. It guarantees that variables are used only when necessary, hence improving code readability and maintainability. So, the next time you begin on a Python coding adventure, keep these scope distinctions in mind for a more efficient and productive development experience.

Recursion

When exploring the field of Python programming, one comes across the fascinating idea of recursion in productive functions. Consider a function calling itself in an infinite cycle of self-discovery. Let's dispel some of the mystery behind this wonderful phenomenon.

In Python, a fruitful function returns a value. When this approach is combined with recursion, it results in a compelling dance of logic and iteration. Consider a function that not only completes a task but also generates additional instances of itself to contribute.

The beauty is in simplicity. Consider a case in which a function takes a recursive path rather than a linear one. It ingeniously breaks a big problem into smaller, more manageable chunks and solves them progressively. Each recursive call adds a layer to the story until the conclusion is revealed.

Syntax highlighting is like shining a spotlight on code, making it look nice and easy to read. The `Editor.copyWithSyntaxHighlighting` acts like a director, ensuring the code runs smoothly.

In Python, recursion in fruitful functions is not just a programming technique; it's a narrative that unfolds with every recursive call, revealing the elegance of a well-crafted story in code. Take on the enchantment and let your functions tell tales of computation in Python's unique recursive language.

Strings in Python

A [string](#) is just a sequence of characters. It is among the most popular [data types in Python](#). It can be created simply by enclosing characters in quotes.

Python provides a rich set of operators, functions, and methods for working with strings, which helps access and extract portions of strings and manipulate and modify string data.

What is a string in python?

Technically speaking, an **immutable data sequence** is known as a string in Python. In simple words, as discussed in the case of a crossword, a Python string is nothing but an array of characters, but a computer does not understand characters. It only understands the language of 0's and 1's.

So these characters are converted to a number so the computer can understand them clearly. This conversion is known as encoding. ASCII and Unicode are some of the popular encodings used.

To put it simply, Python strings are a sequence of **Unicode characters**.

Creating a String in Python

A string in Python can be easily created using single, double, or even triple quotes. The characters that form the string are enclosed within any of these quotes.

Note: Triple quotes are generally used when we are working with multiline strings in Python and docstring in Python.

Let's take a look at an example where we will create strings using the three types of quotes –

Code:

```
single_string='Hello'
double_string="Hello World"
triple_string="""Welcome to Scaler"""
print(single_string)
print(double_string)
print(triple_string)
```

Output:

```
Hello
Hello World
Welcome to Scaler
```

Reassigning Strings in Python

Since strings in Python are immutable, we can update the values of strings simply by reassigning these strings with new values. The strings cannot be partially replaced, so it is always replaced completely with a new string. Let's see how that's done –

Code:

```
message="Welcome to Scaler"
print(message)
message="Hello World" #reassigning a new string
```

```
print(message)
```

Output:

```
Welcome to Scaler  
Hello World
```

From the output, it is evident that the string variable message was reassigned with a new string, which gets printed to the console in the last line.

How to access characters in a python string?

As we know, a string comprises a sequence of characters. Thus, in Python, we have some techniques to reference or index a particular character in a string.

The two primary ways of accessing characters of a string in Python are-

1. Indexing

The length of a string can be easily calculated using the [len\(\) function](#). The length of a string provides us with a range of the string. In python, we can perform indexing on any string as long as it is within the range. The index is always referred to as an integer. Using any other data type would give rise to a TypeError.

Indexing can either be positive or negative. Positive indexing starts from 0 and indexing from the beginning of the string. On the other hand, negative indexing starts from -1, and indexing starts from the end of the string.



Come and explore how indexing works by looking at the following example –

```
message="Welcome to Scaler"  
len_str= len(message)  
print(len_str)  
print(message[4]) # positive indexing within the range
```

```
print(message[-2]) #negative indexing within the range
print(message[33]) #indexing outside the range
```

Output –

```
17
o
e
Traceback (most recent call last) :
  File "main.py", line 6, in <module>
    print (message [33]) #indexing outside the range
IndexError: string index out of range
```

Using the len() function, we calculated the length of the string variable message. We performed positive and negative indexing on this Python string. The thing to note is that we got an IndexError when the index value was way beyond the range of the string.

2. Slicing

Unlike indexing, slicing returns a range of characters using the slicing syntax.

The syntax looks like this –

```
str[start_index: end_index]
```

While performing slicing operations, one thing to remember is that the start index value is always included while that of the last index is always excluded.

Slicing can also be negative or positive, just like indexing in Python!

Let's consider the following examples to understand this better –

Code:

```
message="Welcome to Scaler"
print(message[2:5]) # positive slicing
print(message[-10:-2]) #negative slicing
print(message[:5]) #slicing from the start
print(message[2:]) #slicing to the end
```

Output:

```
lco
  to scal
welco
lcome to Scaler
```

So if we take a close look at the positive slicing performed by us, we can see that the character at the 2nd index is included, whereas at the 5th index is excluded, provided the indexing starts from the 0th position.

How to delete a string?

We have established that strings are immutable. In simple words, it means that once a string is assigned, we cannot make any changes to it. We cannot remove or delete any character of the string.

What we can do is delete the Python string entirely. We can do so by using the del keyword.

The following code snippet shows how we can do that –

```
message="Welcome to Scaler"
print(message)
del message
print(message)
```

The output is –

```
Welcome to Scaler
Traceback (most recent call last) :
  File "main.py", line 4, in <module>
    print (message)
NameError: name 'message' is not defined
```

We can see that before we used del, the string was printed to the console. But as soon as we used the del keyword, the string variable was successfully deleted. Hence, the print function gives us an error.

String Operators in Python

Arithmetic operators do not work on strings. But, we do have certain operators for performing special operations on strings in Python.

Let's check out some of them –

Operator	Description
+	It is used to concatenate two strings. It appends the second string to the first string.
*	It concatenates multiple copies of the same string. It is basically a repetition operator.
[]	It is used for indexing. The value put between this operator returns the character at the given index.
[:]	It is used for slicing. It returns the sub-strings after slicing as per the given range.
in	It returns true if the character is present in the given string.
not in	It returns true if the character is not present in the given string.

Example Program of String Operators in Python

Now that we have learned some Python string operators let's get coding and see how they work –

Code:

```
message1="Hello World!"
message2="Welcome to Scaler"
print(message1+message2) # + operator
print(message1*3) # * operator
print(message1[6]) # [] operator
print(message2[0:7]) # [:] operator
str1="Hello"
if str1 in message1:# in operator
    print("It is a present!")
if str1 not in message2:# not in operator
    print("It is not present!")
```

Output:

```
Hello World!Welcome to Scaler
Hello World!Hello World!Hello World!
w
Welcome
It is a present!
It is not present!
```